Phase 5: Basic Read, Update, and Aggregate Operations in Array ADT

1. Big Idea of Phase 5

Phase 5 teaches simple but very important Array ADT operations.

In earlier phases, you learned how to:

1. Store array data using `A`, `size`, and `length`.
2. Add and remove values using append, insert, and delete.
3. Search values using linear search.
4. Search sorted arrays faster using binary search.

Now Phase 5 focuses on basic operations that answer questions like:

```
What value is stored at this index?
Can I change the value at this index?
What is the largest value?
What is the smallest value?
What is the sum of all values?
What is the average of all values?
```

The main operations in Phase 5 are:

- `get`
- `set`
- `max`
- `min`
- iterative `sum`
- recursive `sum`
- `average`

These operations are called **basic read, update, and aggregate operations**.

Simple idea:

```
Phase 5 = read one value, update one value, or calculate something from all
values.
```

---

2. Quick Review: Array ADT Structure

The fixed-size Array ADT structure is usually written like this:

```
struct Array {
    int A[20];
    int size;
    int length;
};
```

Meaning:

Part	Meaning
`A`	The actual array storage
`size`	Total capacity of the array
`length`	Number of valid elements currently stored

Example:

```
struct Array arr = {{2, 4, 6, 8}, 20, 4};
```

This means:

```
A = [2, 4, 6, 8, _, _, _, ...]
size = 20
length = 4
```

Only these indexes are valid:

```
0, 1, 2, 3
```

The last valid index is:

```
length - 1
```

So here:

```
length - 1 = 4 - 1 = 3
```

Index `4` is not a valid element yet. It is the first free slot.

---

3. The Most Important Rule in Phase 5

For operations that access an existing element, the valid index condition is:

```
index >= 0 && index < length
```

Inside a function that receives the structure by value:

```
index >= 0 && index < arr.length
```

Inside a function that receives a pointer:

```
index >= 0 && index < arr->length
```

This rule is used by:

- `get`
- `set`

It is also the same kind of rule used by operations that access existing values.

Why not `index <= length`?

Because `index == length` is the first free slot, not an existing value.

Example:

```
A = [2, 4, 6, 8]
length = 4
```

Valid indexes:

```
0, 1, 2, 3
```

Invalid index:

```
4
```

Why?

Because index `4` is outside the valid logical array.

Important comparison:

Operation	Valid condition	Why
Insert	`0 <= index <= length`	Insert can happen at the end
Get / Set	`0 <= index < length`	They can access only existing values

Simple memory:

```
Insert can use the first free slot.
Get and set cannot.
```

---

4. Get Operation

4.1 What Get Means

`get` returns the value stored at a specific index.

Example:

```
A = [2, 4, 6, 8]
```

Call:

```
get(arr, 2)
```

Index `2` contains:

```
6
```

So the function returns:

```
6
```

4.2 Get Code

```
int get(struct Array arr, int index) {
    if (index >= 0 && index < arr.length) {
        return arr.A[index];
    }
    return -1;
}
```

4.3 Explanation of Get Code

```
int get(struct Array arr, int index)
```

The function receives:

- `arr`: the Array ADT
- `index`: the position we want to read

```
if (index >= 0 && index < arr.length)
```

This checks whether the index is valid.

```
return arr.A[index];
```

If the index is valid, return the value stored at that index.

```
return -1;
```

If the index is invalid, return `-1`.

4.4 Get Dry Run

Array:

```
A = [2, 4, 6, 8]
length = 4
```

Call:

```
get(arr, 2)
```

Trace:

Step	Check	Result
1	Is `2 >= 0`?	Yes
2	Is `2 < 4`?	Yes
3	Return `A[2]`	`6`

Final result:

```
6
```

Another call:

```
get(arr, 9)
```

Trace:

Step	Check	Result
1	Is `9 >= 0`?	Yes
2	Is `9 < 4`?	No
3	Return invalid result	`-1`

Final result:

```
-1
```

4.5 Why Get Uses Pass-by-Value

`get` only reads the array.

It does not change:

- array values
- `size`
- `length`

So it can receive the structure by value:

```
int get(struct Array arr, int index)
```

4.6 Get Complexity

`get` directly jumps to the index:

```
arr.A[index]
```

It does not use a loop.

So:

Complexity type	Result
Time	`O(1)`
Space	`O(1)`

Simple memory:

```
Get is O(1) because array index access is direct.
```

---

5. Set Operation

5.1 What Set Means

`set` changes the value at a specific index.

Example:

```
A = [2, 4, 6, 8]
```

Call:

```
set(&arr, 1, 20)
```

This means:

```
Change the value at index 1 to 20.
```

Before:

```
[2, 4, 6, 8]
```

After:

```
[2, 20, 6, 8]
```

5.2 Set Code

```
void set(struct Array *arr, int index, int x) {
    if (index >= 0 && index < arr->length) {
        arr->A[index] = x;
    }
}
```

5.3 Explanation of Set Code

```
void set(struct Array *arr, int index, int x)
```

The function receives:

- `arr`: address of the Array ADT
- `index`: the position to update
- `x`: the new value

```
if (index >= 0 && index < arr->length)
```

This checks whether the index is valid.

```
arr->A[index] = x;
```

This replaces the old value with the new value.

5.4 Why Set Uses Pass-by-Address

`set` modifies the real array.

If we passed the structure by value, the function would modify only a copy.

So we pass the address:

```
void set(struct Array *arr, int index, int x)
```

Because `arr` is a pointer, we use the arrow operator:

```
arr->A[index]
arr->length
```

Not:

```
arr.A[index]
arr.length
```

5.5 Set Dry Run

Array:

```
A = [2, 4, 6, 8]
length = 4
```

Call:

```
set(&arr, 0, 15)
```

Trace:

Step	Action	Array
Start	Original array	`[2, 4, 6, 8]`
1	Check `0 >= 0 && 0 < 4`	valid
2	Set `A[0] = 15`	`[15, 4, 6, 8]`

Final array:

```
[15, 4, 6, 8]
```

Invalid call:

```
set(&arr, 9, 15)
```

Since `9 < 4` is false, the function does nothing.

5.6 Set Complexity

`set` directly overwrites one value.

No loop is used.

Complexity type	Result
Time	`O(1)`
Space	`O(1)`

Simple memory:

```
Set is O(1) because it directly updates one index.
```

---

6. Max Operation

6.1 What Max Means

`max` finds the largest value in the array.

Example:

```
A = [8, 3, 9, 15, 6]
```

Largest value:

```
15
```

So `max(arr)` should return:

```
15
```

6.2 Main Idea of Max

Start by assuming the first element is the largest:

```
max = arr.A[0];
```

Then scan the rest of the array.

If any value is greater than the current `max`, update `max`.

Simple process:

1. Assume `A[0]` is the maximum.
2. Compare `A[1]`, `A[2]`, ..., `A[length - 1]`.
3. If a bigger value is found, update `max`.
4. Return `max`.

6.3 Max Code

```
int max(struct Array arr) {
    int max = arr.A[0];

    for (int i = 1; i < arr.length; i++) {
        if (arr.A[i] > max) {
            max = arr.A[i];
        }
    }

    return max;
}
```

6.4 Explanation of Max Code

```
int max = arr.A[0];
```

This assumes the first valid element is the largest.

```
for (int i = 1; i < arr.length; i++)
```

The loop starts from index `1` because index `0` has already been used.

```
if (arr.A[i] > max)
```

If the current value is bigger than `max`, update `max`.

```
return max;
```

Return the largest value after scanning all valid elements.

6.5 Max Dry Run

Array:

```
A = [8, 3, 9, 15, 6]
length = 5
```

Initial:

```
max = A[0] = 8
```

Trace:

Step	`i`	`A[i]`	Comparison	`max`
Start	-	8	Assume first value is max	8
1	1	3	`3 > 8` ? No	8
2	2	9	`9 > 8` ? Yes	9
3	3	15	`15 > 9` ? Yes	15

Step	`i`	`A[i]`	Comparison	`max`
4	4	6	`6 > 15` ? No	15

Final result:

```
15
```

6.6 Why Max Uses Pass-by-Value

`max` only reads the array.

It does not modify any value.

So it can receive the structure by value:

```c
int max(struct Array arr)
```

6.7 Max Complexity

To know the maximum value, the function must check all valid values.

So:

Complexity type	Result
Time	`O(N)`
Space	`O(1)`

Simple memory:

```
Max is O(N) because it scans the array.
```

---

7. Min Operation

7.1 What Min Means

`min` finds the smallest value in the array.

Example:

```
A = [8, 3, 9, 15, 6]
```

Smallest value:

```
3
```

So `min(arr)` should return:

```
3
```

7.2 Main Idea of Min

`min` works like `max`, but it looks for smaller values.

Process:

1. Assume `A[0]` is the minimum.
2. Scan from index `1` to `length - 1`.
3. If a smaller value is found, update `min`.
4. Return `min`.

7.3 Min Code

```
int min(struct Array arr) {
    int min = arr.A[0];

    for (int i = 1; i < arr.length; i++) {
        if (arr.A[i] < min) {
            min = arr.A[i];
        }
    }

    return min;
}
```

7.4 Min Dry Run

Array:

```
A = [8, 3, 9, 15, 6]
length = 5
```

Initial:

```
min = A[0] = 8
```

Trace:

Step	`i`	`A[i]`	Comparison	`min`
Start	-	8	Assume first value is min	8
1	1	3	`3 < 8` ? Yes	3
2	2	9	`9 < 3` ? No	3
3	3	15	`15 < 3` ? No	3
4	4	6	`6 < 3` ? No	3

Final result:

```
3
```

7.5 Min Complexity

`min` must scan all valid elements.

So:

Complexity type	Result
Time	`O(N)`
Space	`O(1)`

Simple memory:

```
Min is O(N) because it scans the array.
```

---

8. Sum Operation

8.1 What Sum Means

`sum` adds all valid values in the array.

Example:

```
A = [2, 4, 6, 8]
```

Sum:

```
2 + 4 + 6 + 8 = 20
```

So `sum(arr)` should return:

```
20
```

There are two versions:

1. Iterative sum
2. Recursive sum

---

9. Iterative Sum

9.1 What Iterative Sum Means

Iterative sum uses a loop.

It starts with:

```
total = 0
```

Then it adds each valid element one by one.

9.2 Iterative Sum Code

```
int sum(struct Array arr) {
    int total = 0;

    for (int i = 0; i < arr.length; i++) {
        total = total + arr.A[i];
    }

    return total;
}
```

9.3 Explanation of Iterative Sum Code

```
int total = 0;
```

The total starts at zero because no values have been added yet.

```
for (int i = 0; i < arr.length; i++)
```

Loop through all valid elements.

```
total = total + arr.A[i];
```

Add the current value into `total`.

```
return total;
```

Return the final sum.

9.4 Iterative Sum Dry Run

Array:

```
A = [2, 4, 6, 8]
length = 4
```

Initial:

```
total = 0
```

Trace:

Step	`i`	`A[i]`	New total
Start	-	-	0
1	0	2	2
2	1	4	6
3	2	6	12
4	3	8	20

Final result:

```
20
```

9.5 Iterative Sum Complexity

The loop visits every valid element once.

So:

Complexity type	Result
Time	`O(N)`
Space	`O(1)`

Simple memory:

```
Iterative sum is O(N) time and O(1) space.
```

---

10. Recursive Sum

10.1 What Recursive Sum Means

Recursive sum calculates the sum by calling itself with a smaller index.

Instead of looping forward, it usually starts from the last valid index:

```
arr.length - 1
```

Example:

```
A = [2, 4, 6, 8]
length = 4
```

Last valid index:

```
length - 1 = 3
```

So we call:

```
RSum(arr, 3)
```

10.2 Recursive Sum Code

```
int RSum(struct Array arr, int n) {
    if (n < 0) {
        return 0;
    }

    return RSum(arr, n - 1) + arr.A[n];
}
```

10.3 Meaning of `n`

In this function, `n` means the current index being added.

If we call:

```
RSum(arr, 3)
```

It means:

```
Add values from index 0 to index 3.
```

10.4 Recursive Sum Expansion

Array:

```
A = [2, 4, 6, 8]
```

Call:

```
RSum(arr, 3)
```

Expansion:

```
RSum(arr, 3) = RSum(arr, 2) + A[3]
RSum(arr, 2) = RSum(arr, 1) + A[2]
RSum(arr, 1) = RSum(arr, 0) + A[1]
RSum(arr, 0) = RSum(arr, -1) + A[0]
RSum(arr, -1) = 0
```

Now substitute values:

```
RSum(arr, -1) = 0
RSum(arr, 0) = 0 + 2 = 2
RSum(arr, 1) = 2 + 4 = 6
RSum(arr, 2) = 6 + 6 = 12
RSum(arr, 3) = 12 + 8 = 20
```

Final result:

```
20
```

10.5 Base Case

The base case is:

```
if (n < 0) {
    return 0;
}
```

This means:

```
If there are no more elements to add, return 0.
```

Without the base case, recursion would not stop correctly.

10.6 Recursive Sum Complexity

Recursive sum still adds every valid element.

So the time is:

```
O(N)
```

But recursive sum also uses the call stack.

Each recursive call waits for the next call to finish.

So the space is:

```
O(N)
```

Summary:

Version	Time	Space
Iterative sum	`O(N)`	`O(1)`
Recursive sum	`O(N)`	`O(N)`

Simple memory:

```
Recursive sum is not faster than iterative sum.
It uses extra stack space.
```

---

11. Average Operation

11.1 What Average Means

Average means:

```
sum / number of values
```

For an Array ADT:

```
average = sum / length
```

Example:

```
A = [2, 4, 6, 8]
```

Sum:

```
2 + 4 + 6 + 8 = 20
```

Length:

```
4
```

Average:

```
20 / 4 = 5
```

11.2 Average Code

```c
float average(struct Array arr) {
    int total = sum(arr);
    return (float)total / arr.length;
}
```

11.3 Explanation of Average Code

```
int total = sum(arr);
```

First, calculate the total using the `sum` function.

```
return (float)total / arr.length;
```

Then divide the total by the number of valid elements.

The result is returned as a `float` because average can contain decimal values.

11.4 Why `(float)` Casting Is Needed

In C, integer division removes the decimal part.

Example:

```
5 / 2
```

Result:

```
2
```

Not:

```
2.5
```

Why?

Because both `5` and `2` are integers.

To keep the decimal part, cast one value to `float`:

```
(float)5 / 2
```

Result:

```
2.5
```

So average should use:

```
(float)total / arr.length
```

This prevents decimal values from being lost.

11.5 Average Dry Run

Array:

```
A = [2, 4, 6, 9]
length = 4
```

Sum:

```
2 + 4 + 6 + 9 = 21
```

Average:

```
21 / 4 = 5.25
```

Without float casting:

```
21 / 4 = 5
```

With float casting:

```
(float)21 / 4 = 5.25
```

11.6 Average Complexity

Average calls `sum(arr)`.

Since `sum` scans all valid values, average is also:

Complexity type	Result
Time	`O(N)`
Space	`O(1)` auxiliary space

Simple memory:

```
Average is O(N) because it depends on sum.
```

---

12. Pass-by-Value vs Pass-by-Address in Phase 5

This is one of the most important C ideas in Phase 5.

12.1 Pass-by-Value

A function can receive the structure by value if it only reads the array.

Examples:

```c
int get(struct Array arr, int index)
int max(struct Array arr)
int min(struct Array arr)
int sum(struct Array arr)
int RSum(struct Array arr, int n)
float average(struct Array arr)
```

These functions do not permanently modify the original array.

12.2 Pass-by-Address

A function must receive the address if it modifies the real array.

Example:

```c
void set(struct Array *arr, int index, int x)
```

`set` changes the value inside the original array, so it needs a pointer.

12.3 Dot vs Arrow Operator

When the function receives a normal structure variable, use dot:

```
arr.length
arr.A[i]
```

When the function receives a pointer to a structure, use arrow:

```
arr->length
arr->A[i]
```

Summary:

Operation	Changes array?	Parameter style	Access style
`get`	No	`struct Array arr`	`arr.length`
`set`	Yes	`struct Array *arr`	`arr->length`
`max`	No	`struct Array arr`	`arr.length`
`min`	No	`struct Array arr`	`arr.length`
`sum`	No	`struct Array arr`	`arr.length`
`RSum`	No	`struct Array arr`	`arr.length`
`average`	No	`struct Array arr`	`arr.length`

Simple memory:

```
Read only? Pass by value.
Modify data? Pass by address.
```

---

13. Complete Phase 5 Program

```c
#include <stdio.h>

struct Array {
    int A[20];
    int size;
```

```
    int length;
};

void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}

int get(struct Array arr, int index) {
    if (index >= 0 && index < arr.length) {
        return arr.A[index];
    }
    return -1;
}

void set(struct Array *arr, int index, int x) {
    if (index >= 0 && index < arr->length) {
        arr->A[index] = x;
    }
}

int max(struct Array arr) {
    int max = arr.A[0];

    for (int i = 1; i < arr.length; i++) {
        if (arr.A[i] > max) {
            max = arr.A[i];
        }
    }

    return max;
}

int min(struct Array arr) {
    int min = arr.A[0];

    for (int i = 1; i < arr.length; i++) {
        if (arr.A[i] < min) {
            min = arr.A[i];
        }
    }

    return min;
}

int sum(struct Array arr) {
```

```
    int total = 0;

    for (int i = 0; i < arr.length; i++) {
        total = total + arr.A[i];
    }

    return total;
}

int RSum(struct Array arr, int n) {
    if (n < 0) {
        return 0;
    }

    return RSum(arr, n - 1) + arr.A[n];
}

float average(struct Array arr) {
    int total = sum(arr);
    return (float)total / arr.length;
}

int main() {
    struct Array arr = {{2, 4, 6, 8, 10}, 20, 5};

    printf("Original array:\n");
    display(arr);

    printf("Get index 2: %d\n", get(arr, 2));

    set(&arr, 1, 15);
    printf("After setting index 1 to 15:\n");
    display(arr);

    printf("Max: %d\n", max(arr));
    printf("Min: %d\n", min(arr));
    printf("Sum: %d\n", sum(arr));
    printf("Recursive Sum: %d\n", RSum(arr, arr.length - 1));
    printf("Average: %.2f\n", average(arr));

    return 0;
}
```

---

14. Program Walkthrough

Starting array:

```
[2, 4, 6, 8, 10]
```

Values:

```
size = 20
length = 5
```

Step 1: Display

```
display(arr);
```

Output:

```
2 4 6 8 10
```

Step 2: Get Index 2

```
get(arr, 2)
```

Index `2` contains:

```
6
```

Output:

```
Get index 2: 6
```

Step 3: Set Index 1 to 15

```
set(&arr, 1, 15);
```

Before:

```
[2, 4, 6, 8, 10]
```

After:

```
[2, 15, 6, 8, 10]
```

Step 4: Max

```
max(arr)
```

Array:

```
[2, 15, 6, 8, 10]
```

Largest value:

```
15
```

Step 5: Min

```
min(arr)
```

Smallest value:

```
2
```

Step 6: Sum

```
sum(arr)
```

Calculation:

```
2 + 15 + 6 + 8 + 10 = 41
```

Result:

```
41
```

Step 7: Recursive Sum

```
RSum(arr, arr.length - 1)
```

This means:

```
RSum(arr, 4)
```

It also returns:

```
41
```

Step 8: Average

```
average(arr)
```

Calculation:

```
41 / 5 = 8.2
```

Output:

```
8.20
```

---

15. Expected Output

```
Original array:
2 4 6 8 10
Get index 2: 6
After setting index 1 to 15:
2 15 6 8 10
Max: 15
Min: 2
```

```
Sum: 41
Recursive Sum: 41
Average: 8.20
```

---

16. Complexity Summary

Let:

```
N = arr.length
```

That means `N` is the number of valid elements.

Operation	Time Complexity	Space Complexity	Why
`get`	`O(1)`	`O(1)`	Direct index access
`set`	`O(1)`	`O(1)`	Direct overwrite
`max`	`O(N)`	`O(1)`	Scans all valid values
`min`	`O(N)`	`O(1)`	Scans all valid values
iterative `sum`	`O(N)`	`O(1)`	Adds all values using a loop
recursive `sum`	`O(N)`	`O(N)`	Adds all values using recursion and call stack
`average`	`O(N)`	`O(1)` auxiliary	Calls `sum`, which scans all values

Important comparison:

```
get and set are O(1) because they access one index directly.
max, min, sum, and average are O(N) because they need to scan the array.
```

---

17. Important Real-Code Warning About Empty Arrays

The beginner code assumes the array has at least one valid element.

These functions use:

```
arr.A[0]
```

Examples:

```
int max = arr.A[0];
int min = arr.A[0];
```

This is safe only if:

```
arr.length > 0
```

If:

```
arr.length == 0
```

Then there is no valid `A[0]`.

So in stronger real-world code, you should check for an empty array before calling:

- `max`
- `min`
- `average`

Example safer idea:

```
if (arr.length == 0) {
    printf("Array is empty.\n");
}
```

For beginner learning, the examples use non-empty arrays to keep the logic simple.

---

18. Common Beginner Mistakes in Phase 5

Mistake 1: Using `index <= length` for get or set

Wrong:

```
if (index >= 0 && index <= arr.length)
```

Correct:

```
if (index >= 0 && index < arr.length)
```

Reason:

```
index == length is not an existing element.
```

Mistake 2: Forgetting that set modifies the array

Wrong function style:

```
void set(struct Array arr, int index, int x)
```

This changes only a copy.

Correct:

```
void set(struct Array *arr, int index, int x)
```

Reason:

```
set must modify the real array.
```

Mistake 3: Using dot instead of arrow inside set

Wrong:

```
arr.length
arr.A[index]
```

Correct:

```
arr->length
arr->A[index]
```

Reason:

`arr` is a pointer in the `set` function.

Mistake 4: Starting max or min from index 0 in the loop

This is not always wrong, but it is unnecessary.

Better pattern:

```
int max = arr.A[0];
for (int i = 1; i < arr.length; i++)
```

Reason:

Index `0` is already used as the starting answer.

Mistake 5: Forgetting to initialize sum

Wrong:

```
int total;
```

Correct:

```
int total = 0;
```

Reason:

An uninitialized variable may contain garbage.

Mistake 6: Returning `-1` too early from get-style logic

For `get`, returning `-1` after checking validity is okay.

But for scanning functions like `max`, `min`, and `sum`, do not return before the scan is complete.

Mistake 7: Forgetting the recursive base case

Wrong idea:

```
return RSum(arr, n - 1) + arr.A[n];
```

Without a base case, recursion does not stop correctly.

Correct:

```
if (n < 0) {
    return 0;
}
```

Mistake 8: Forgetting `(float)` in average

Wrong:

```
return total / arr.length;
```

Correct:

```
return (float)total / arr.length;
```

Reason:

Without `(float)`, C may do integer division and remove decimals.

Mistake 9: Calling max, min, or average on an empty array

Unsafe:

```
arr.length = 0;
max(arr);
```

Reason:

There is no valid `A[0]` when the array is empty.

---

19. Beginner Memory Tricks

For Get

```
Get = go to index and return value.
```

For Set

```
Set = go to index and replace value.
```

For Max

```
Max = start with A[0], keep the bigger value.
```

For Min

```
Min = start with A[0], keep the smaller value.
```

For Sum

```
Sum = start at 0 and add every valid element.
```

For Recursive Sum

```
Recursive sum = sum previous indexes + current value.
```

For Average

```
Average = sum / length.
```

For Complexity

```
One index only = O(1)
Scan all values = O(N)
Recursion over all values = O(N) time and O(N) stack space
```

---

20. Operation Summary Table

Operation	Meaning	Changes array?	Main action	Time
`get`	Return value at index	No	Direct access	`O(1)`
`set`	Replace value at index	Yes	Direct overwrite	`O(1)`
`max`	Find largest value	No	Scan and keep largest	`O(N)`
`min`	Find smallest value	No	Scan and keep smallest	`O(N)`
iterative `sum`	Add all values	No	Loop through valid values	`O(N)`
recursive `sum`	Add all values recursively	No	Recursive calls	`O(N)`
`average`	Find mean value	No	Sum divided by length	`O(N)`

---

21. Phase 5 Checkpoint Questions

You understand Phase 5 when you can answer these without looking:

1. What does `get` do?
2. What does `set` do?
3. Why does `get` use `index < length`?
4. Why is `index == length` invalid for `get` and `set`?
5. Why does `set` need a pointer?
6. When do we use `arr.length`?
7. When do we use `arr->length`?
8. Why is `get` `O(1)`?
9. Why is `set` `O(1)`?

10. How does `max` find the largest value?
11. Why does `max` start with `A[0]`?
12. Why does the `max` loop start from index `1`?
13. How does `min` find the smallest value?
14. Why are `max` and `min` `O(N)`?
15. How does iterative sum work?
16. Why does sum start with `total = 0`?
17. What does `n` mean in recursive sum?
18. What is the base case of recursive sum?
19. Why does recursive sum use `O(N)` space?
20. How is average calculated?
21. Why is `(float)` needed in average?
22. Why is average `O(N)`?
23. What problem happens if max or min is called on an empty array?
24. Which Phase 5 operation modifies the array?
25. Which Phase 5 operations only read the array?

---

22. Phase 5 Final Summary

Phase 5 teaches basic Array ADT operations for reading, updating, and calculating values.

The main operations are:

- `get`: returns the value at an index
- `set`: changes the value at an index
- `max`: finds the largest value
- `min`: finds the smallest value
- iterative `sum`: adds values using a loop
- recursive `sum`: adds values using recursion
- `average`: calculates sum divided by length

The most important index rule is:

```
index >= 0 && index < length
```

The most important passing rule is:

```
Read only = pass by value
Modify array = pass by address
```

The most important complexity rule is:

```
Direct index access = O(1)
Scanning all values = O(N)
Recursive scan = O(N) time and O(N) stack space
```

---

23. One-Line Final Memory

Phase 5 is about using valid indexes to read one value with `get`, update one value with `set`, and scan the array to calculate `max`, `min`, `sum`, and `average`.